

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

КАФЕДРА № 24

ЛАБОРАТОРНАЯ РАБОТА
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

канд. тех. наук, доцент

Саенко В.И.

должность, уч. степень, звание

подпись, дата

инициалы, фамилия

ЛАБОРАТОРНАЯ РАБОТА № 4

Ссылки и указатели (Pointers)

по дисциплине: Алгоритмизация и Программирование

(очная форма обучения направление 12.03.04 Биотехнические системы и технологии)

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №

2346

Исаева Е.О.

номер группы

подпись, дата

инициалы, фамилия

Санкт-Петербург 2025

Содержание

Тема	3
Цель работы	3
Пояснение к отчету по лабораторной работе	3
Часть 1. Ссылки.	3
Цель.	3
Результаты упражнения 1.3. Использование ссылок для оптимизации цикла.	3
Часть 2. Указатели.	4
Цель.	4
Результаты упражнения 2.1. Пример использования указателей.	4
Результаты упражнения 2.2. Присвоение указателю адрес объекта.	5
Результаты упражнения 2.3. Приобретение значения объекта по указателю.	6
Результаты упражнения 2.5. Проверка использования ссылок на указатели.	7
Результаты упражнения 2.6. Согласование типов данных для указателей.	9
Результаты упражнения 2.8. Указатели и арифметические операции.	10
Результаты упражнения 2.9. Константы и указатели.	12
Часть 3. Указатели и массивы.	14
Результаты упражнения 3.1. Проверка использования имени массива, как указателя.	14
Результаты упражнения 3.2. Указатели на массивы.	14
Результаты упражнения 3.4. Перебор элементов многомерного массива с использованием указателей.	15
Контрольная задача.	16
Цель	16
Выбор варианта	16
Задание	17
Листинг программ.	17
Итоговая таблица.	28
Выводы по результатам контрольного задания	28
Вывод.	28

Тема

Ссылки и указатели (Pointers).

Цель работы

Формирование навыков по базовым правилам использования ссылок и указателей на языке c++.

Пояснение к отчету по лабораторной работе

Лабораторная работа выполнялась в VSC (Visual Studio Code).

Часть 1. Ссылки.

Цель.

Цель – формирование навыка по базовым правилам использования ссылок на языке c++.

Результаты упражнения 1.3. Использование ссылок для оптимизации цикла.

На рисунке 1 приведен пример программы с использованием ссылки для оптимизации цикла. В данном случае изначально был задан массив `num[] {1,2,3,4,5}`, и было произведено три примера вывода массива. В первом случае использовался классический пример вывода, только внутри цикла было добавлено условие, чтобы выводился не только введенный массив, но и квадрат каждого числа в массиве. Во втором случае использовалась ссылка на массив, и цикл с условием (`m=m*2m-45`) был вынесен отдельно, а вывод массив уже был через ссылку, отдельным циклом, что позволило программе работать быстрее, чем в первом случае (тут значения выбираются по адресам). В третьем случае (блоке) использовалась ссылка для вывода массива, у которого уже вместо первоначальных значений – значения условия `m=m*2m-45`.

```
Dr: modelirovanie 2020 > algprog > lab4 > 3.cpp > main()
1 #include <iostream>
2
3 int main()
4 {
5     int num[] {1, 2, 3, 4, 5};
6
7     // меняем значение переменной n на квадрат элемента массива. Без изменения значений элементов массива
8     // Блок 1
9     std::cout << "Part 1 : "<< "\n";
10
11     for (auto n : num)
12     {
13         std::cout << n << "\t"; // опять из массива
14         n = n * n;
15         std::cout << n << "\t";
16         std::cout << "\n"; // изменение
17     }
18     std::cout << std::endl;
19
20     // Блок 2
21     std::cout << "Part 2 : "<< "\n";
22
23     for (auto& m : num)
24     {
25         m = m * 2 * m - 45;
26     }
27     // все значения элементов массива изменились и стали квадратами
28     // проверяем
29     for (auto m : num)
30     {
31         std::cout << m << "\t";
32     }
33     std::cout << std::endl;
34
35     // Блок 3
36     std::cout << "Part 3 : "<< "\n";
37
38     for (const auto& k : num)
39     {
40         std::cout << k << "\t";
41     }
42     std::cout << std::endl;
43
44     return 0;
45 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

cppdbg 3.exe + - - - ^ x

Part 1 :
1 1
2 4
3 9
4 16
5 25

Part 2 :
-43 -37 -27 -13 5

Part 3 :
-43 -37 -27 -13 5

PS C:\Users\Ekaterina>

Ln 22, Col 5 Spaces: 4 UTF-8 CRLF () C++ Win32

Рисунок 1. Код и результаты компиляции с использованием ссылки для оптимизации цикла.

Часть 2. Указатели.

Цель.

Цель – формирование навыка по базовым правилам использования указателей на языке с++.

Результаты упражнения 2.1. Пример использования указателей.

На рисунке 2 программа с примерами использования указателей. В данном случае, были заданы некоторые переменные (массивы), указатели (которые ни на что не указывают, а также был введен указатель со специальной нулевой константой – nullptr), а также на консоль были выведены указатели, и размерность в байтах заданных переменных и указателей. Размерность указателей – 0, так как значения еще не определены, они «указывают в никуда», а вот у переменных размерность соответствует количеству заданных значений в массиве (каждая переменная соответствует размеру 4 байта).

```
D:\modelirovanie 2020> alprogr > lab4 > @ 4.cpp > main()
1
2 #include <iostream>
3
4 int main()
5 {
6     // Блок 1. Задаем указатели
7     // какие-то переменные
8     int a(10);
9     int b[] {10, 11, 12, 13};
10    double c(10.0);
11
12    // указатели
13    long* p;
14    long* p1();
15    long* p2();
16    float* p4(nullptr); // nullptr - специальная нулевая константа
17    double* p5();
18
19    // вывод на консоль значений указателя (ссылка на адреса). Для p может быть неопределено
20    std::cout << p << '\t';
21    std::cout << p1 << '\t';
22    std::cout << p2 << '\t';
23    std::cout << p4 << '\t';
24    std::cout << p5 << std::endl;
25    std::cout << nullptr << std::endl;
26    std::cout << "checking size of pointers and variables" << std::endl;
27
28    // Блок 2
29    // Определим размеры в байтах для указателей и для переменных
30    // размеры указателя 8 байт, зависит от архитектуры компьютера 32-разр, 64-разр.
31    std::cout << sizeof(p) << '\t';
32    std::cout << sizeof(p1) << '\t';
33    std::cout << sizeof(p4) << '\t';
34    std::cout << sizeof(p5) << std::endl;
35
36    // размеры переменной 4 байт
37    std::cout << sizeof(a) << '\t';
38    std::cout << sizeof(b[0]) << '\t';
39    std::cout << sizeof(b) << '\t';
40    std::cout << sizeof(c) << std::endl;
41
42    return 0;
43 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Екатерина> & "C:\Users\Екатерина\.vscode\extensions\ms-vscode.cpptools-1.25.3-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe" "--stdout=Microsoft-MIEngine-In-ajvzpa0d.jyg" "--stdout=Microsoft-MIEngine-Out-msf4ti4.whh" "--stderr=Microsoft-MIEngine-Error-xw4mfwg2.55c" "--pid=Microsoft-MIEngine-Pid-114glurk.gv1" "--dbgExe=D:\modelirovanie 2020\alprogr\mingw\vcrt64\bin\gdb.exe" "--interpreter=mi"

0 0 0 0

nullptr

checking size of pointers and variables

8 8 8 8

4 4 16 8

PS C:\Users\Екатерина>

Рисунок 2. Код и результаты компиляции программы с использованием указателей и определением размерности для указателей и заданных переменных.

Результаты упражнения 2.2. Присвоение указателю адрес объекта.

На рисунке 3 программа с примером присваиванием адреса объекта указателю.

Надо отметить сразу, что из-за особенностей отладчика, адрес на консоль выводится всегда одинаковый, но если запускать напрямую через консоль, то адреса будут меняться (как только программа написана ее можно запустить только через отладчик, чтобы она собралась, и только после уже можно запускать программу через консоль). В данной программе была произведена инициализация трех переменных с разным типом данных, а также были обозначены указатели, которым был присвоен тот же тип переменных, который соответствует адресу переменной, на которую указывает указатель. После чего указатели были выведены на консоль, в данном случае мы видим адрес первой ячейки (всего их 4, так как переменная, например `int a{10}`, содержит всего лишь 1 число, что соответствует размеру 4 байта, соответственно занимает 4 ячейки памяти), которую занимает выведенная на консоль переменная. Так же в данном коде было проверено автоматическое определение типа переменной

(auto) для указателей, что позволило беспрепятственно вывести адреса заданных переменных. После чего так же был выведен адрес самих указателей, т е место, где находятся указатели.

Тип переменных `int` 1 символ занимает размер 4 байта.

Тип переменных double 1 символ занимает 8 байтов.

Указатель занимает в 64-х разрядной системе занимает 8 байтов (в 32-х разрядной системе 4 байта).

```

1  klog > 0 main
2  int main()
3  {
4      // блок 1. Инициализация переменных и инициализация переменных
5      // Инициализация переменных, инициализация
6      int a[8];
7      int b[3];
8      double c[18.0];
9
10     // блок 2. Инициализация переменных
11     int* p1 (a);
12     int* p2 (b);
13     double* p3 (c);
14
15     // блок 3. Инициализация переменных
16     // Инициализация переменных, инициализация (ссылка на адрес) - для переменных
17     std::cout << p1 << "\n";
18     std::cout << p2 << "\n";
19     std::cout << p3 << "\n";
20
21     // блок 4. Инициализация переменных
22     // Инициализация переменных, инициализация
23     auto* p4(a);
24     auto* p5(b);
25     auto* p6(c);
26
27     std::cout << "auto b и c: " << p5 << "\n";
28     std::cout << p6 << "\n";
29     std::cout << std::endl;
30
31     // блок 5.
32     // Инициализация переменных, инициализация
33     std::cout << "pointer addresses p1, p2, p3: " << p1 << "\n";
34     std::cout << "pointer addresses p4, p5, p6: " << p4 << "\n";
35     std::cout << "pointer addresses p4, p5, p6: " << p5 << "\n";
36     std::cout << "pointer addresses p4, p5, p6: " << p6 << "\n";
37     return 0;
38 }

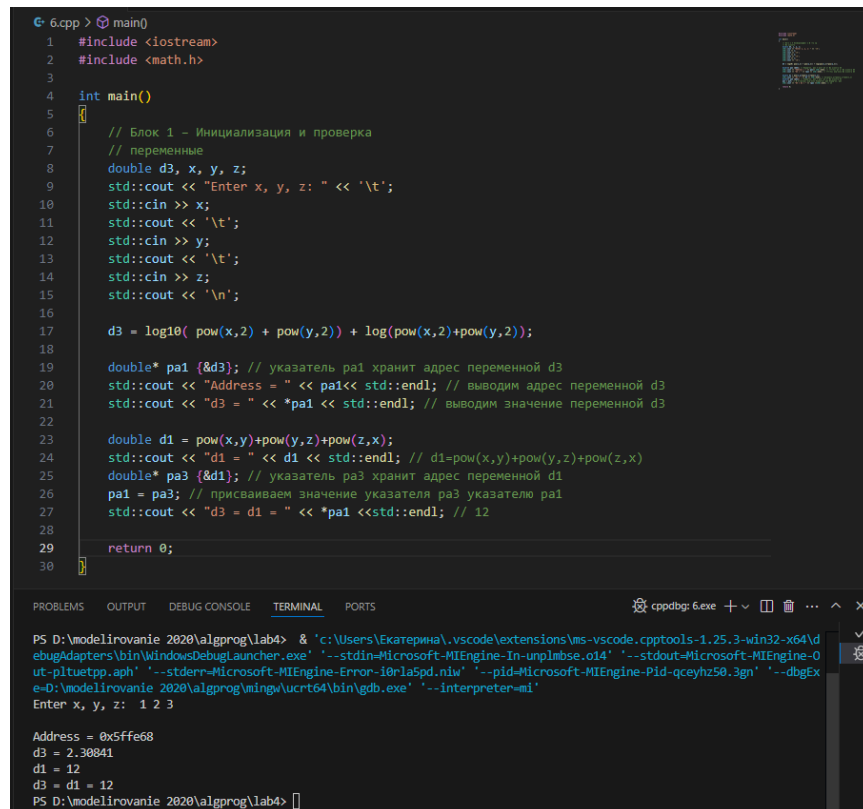
```

Рисунок 3. Код и результаты компиляции программы с присвоением указателю адрес объекта.

Результаты упражнения 2.3. Приобретение значения объекта по указателю.

На рисунке 4 программа с приобретением значения объекта по указателю. В данной программе переменные **x**, **y**, **z** для вычисления операций **(d1 = x^y + y^z + z^x и d3 = lg(x² + y²) + ln(x² + y²))** вводятся с консоли. Так же вводится указатель **p1** который указывает на переменную **d3**, далее производится вывод переменной через указатель, чтобы показать, что изначальное значение переменной соответствует заданной операции **d3**. После чего вводится новая переменная **d1** и указатель, который указывает на данную переменную **pa3**, и выводится на консоль переменная, чтобы показать значение при вычислении операции. В итоговом блоке происходит присваивание значения переменной **d1** переменной **d3** через указатели, и выводится на консоль значение, чтобы показать, что присвоение произошло верно. Вариант

номера задания был рассчитан исходя из номера в журнале (8), что соответствует 3 заданию (d3, d1).



```
6.cpp > main()
1  #include <iostream>
2  #include <math.h>
3
4  int main()
5  {
6      // Блок 1 - Инициализация и проверка
7      // переменные
8      double d3, x, y, z;
9      std::cout << "Enter x, y, z: " << '\t';
10     std::cin >> x;
11     std::cout << '\t';
12     std::cin >> y;
13     std::cout << '\t';
14     std::cin >> z;
15     std::cout << '\n';
16
17     d3 = log10( pow(x,2) + pow(y,2) ) + log(pow(x,2)+pow(y,2));
18
19     double* pa1 (&d3); // указатель pa1 хранит адрес переменной d3
20     std::cout << "Address = " << pa1<< std::endl; // выводим адрес переменной d3
21     std::cout << "d3 = " << *pa1 << std::endl; // выводим значение переменной d3
22
23     double d1 = pow(x,y)+pow(y,z)+pow(z,x);
24     std::cout << "d1 = " << d1 << std::endl; // d1=pow(x,y)+pow(y,z)+pow(z,x)
25     double* pa3 (&d1); // указатель pa3 хранит адрес переменной d1
26     pa1 = pa3; // присваиваем значение указателя pa3 указателю pa1
27     std::cout << "d3 = d1 = " << *pa1 <<std::endl; // 12
28
29     return 0;
30 }
```

PS D:\modelirovanie 2020\algprog\lab4> & 'c:\Users\Ekaterina\.vscode\extensions\ms-vscode.cpptools-1.25.3-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-unplmbse.o14' '--stdout=Microsoft-MIEngine-Output-pltwsttp.aph' '--stderr=Microsoft-MIEngine-Error-10rla5pd.nlw' '--pid=Microsoft-MIEngine-Pid-qceyhiz50.3gn' '--dbgExe=D:\modelirovanie 2020\algprog\mingw\lucrt64\bin\gdb.exe' '--interpreter=mi'

Enter x, y, z: 1 2 3

Address = 0x5ffe68
d3 = 2.30841
d1 = 12
d3 = d1 = 12

PS D:\modelirovanie 2020\algprog\lab4>

Рисунок 4. Код и результаты компиляции программы с присваиванием нового значения другой переменной по указателю.

Результаты упражнения 2.5. Проверка использования ссылок на указатели.

На рисунке 5 программа с проверкой использования ссылок и указателей. Программа состоит из 5 блоков. Изначально задаются две переменные **a {15}** и **b {8}**. После чего создается пустой указатель **p {}**, ссылка на переменную **a (&ref_a {a})** и указатель для переменной **a (aPtr1(&a))**. Далее проводится проверка инициализации переменных, на консоль выводятся заданные: ссылка на переменную, переменная через указатель и адрес переменной через указатель. Во **втором блоке** была создана ссылка на пустой указатель **(&aPtr2 {p})**, после чего через ссылку указателю был присвоен адрес переменной **a** и произведена проверка инициализации, которая была успешна. Далее в **третьем блоке** значение переменной **a** было изменено по адресу **&aPtr2** на значение **55**, таким образом переменной **a** было присвоено значение **55**. В **четвертом блоке** было произведено переназначение адреса

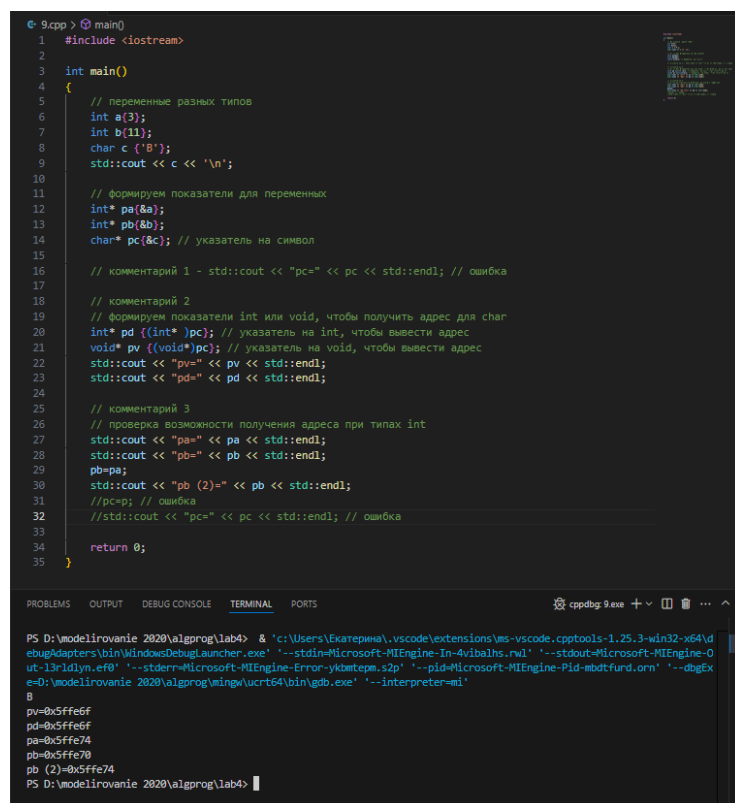
переменной **b**, на который указывает указатель **aPtr2**. После этого было произведена проверка инициализации, которая прошла успешно. В **пятом блоке** снова было произведено переназначение переменной **b**, но уже на который указывает указатель **aPtr1**. Таким образом теперь и при значении выводе переменной через указатель **aPtr2** и **aPtr1** будет соответствовать значению переменной **b = 8**. А вот значение переменной **a** не поменялось относительно последних изменений в **блоке 4**, т.е. осталось равно **55**, потому что присвоение значение адреса через указатель было произведено до переназначения адреса переменной **b**.

```
8.cpp > main()
1  #include <iostream>
2
3  int main()
4  {
5      // комментарий 1 блок 1
6      // просто переменные
7      int a {15};
8      int b {8};
9      // создаем указатели и ссылки
10     int* p{}; // указатель, пустой
11     int &ref_a {a}; //ссылка на переменную a
12     int* aPtr1 {&a}; // указатель для a
13     // проверяем все инициализованные переменные
14     std::cout << " ref_a value= " << ref_a << std::endl; // 15
15     std::cout << " *aPtr1 value= " << *aPtr1 << std::endl; // 15
16     std::cout << " aPtr1 address value= " << aPtr1 << std::endl; // xxx-address-xxx
17     std::cout << "===== " << std::endl;
18     // комментарий 2 блок 2, создаем ссылку на указатель
19     int* &aPtr2 {p}; // ссылка на указатель
20     // используем ссылку: присваиваем адрес переменной a для aPtr2
21     aPtr2 = &a; // через ссылку &aPtr2 указателю p присваивается адрес переменной a
22     std::cout << "p value=" << *p << std::endl; // 15
23     // комментарий 3 блок 3
24     *aPtr2 = 55; // изменяем значение по адресу, на который указывает указатель, т.е. a
25     std::cout << "a value=" << a << std::endl; // 55
26
27     // комментарий 4 блок 4
28     // еще раз переназначение: переназначаем адрес переменной b для aPtr2
29     aPtr2 = &b; // изменяем адрес, на который указывает указатель aPtr2
30     // проверка
31     std::cout << "aPtr2 value=" << *aPtr2 << std::endl; // 8
32     std::cout << "p value=" << *p << std::endl; // 8
33     std::cout << "a value=" << a << std::endl; // 55
34
35     // комментарий 5 блок 5
36     // проверяем эти же действия без ссылки
37     // переназначаем адреса для aPtr1 и проверяем
38     aPtr1 = &b; //
39     std::cout << " *aPtr1 value=" << *aPtr1 << std::endl; // 8
40     std::cout << "a value=" << a << std::endl; // 55
41     // еще раз переназначаем адреса для aPtr1 и проверяем
42     aPtr1 = &a;
43     std::cout << " *aPtr1 value=" << *aPtr1 << std::endl; // 55
44     aPtr1 = &b; //
45     std::cout << " *aPtr1 value=" << *aPtr1 << std::endl; // 8
46
47     return 0;
48 }
```

Рисунок 5. Код программы с проверкой использования ссылок и указателей.

Результаты упражнения 2.6. Согласование типов данных для указателей.

На рисунке 6 программа с присваиванием указателю одного типа значение указателя другого типа. В данной программе приведен пример вывода адреса переменной с типом `char` (символьный) через указатель с типом `int`, а так же с типом `void` (если тип указателя равен `void*`, указатель может указывать на любую переменную, которая не объявлена с помощью ключевого слова или `volatile`). Для того, чтобы произвести вывод адреса с разными типами переменной и указателя, использовался следующий формат записи `int* pd {(int*)pc}`, где внутри фигурных скобок специально введенного дополнительного указателя **pd** задавался новый тип указателя **pc**, который указывает адрес переменной **c** с типом `char` (у указателя **pc** также задан тип `char`). Также в данной программе была произведена проверка возможности получения адреса у указателя и переменной с одним типом данных, а так же было произведено повторение из предыдущего упражнения, переназначение переменных через указатели (**pb=pa**).



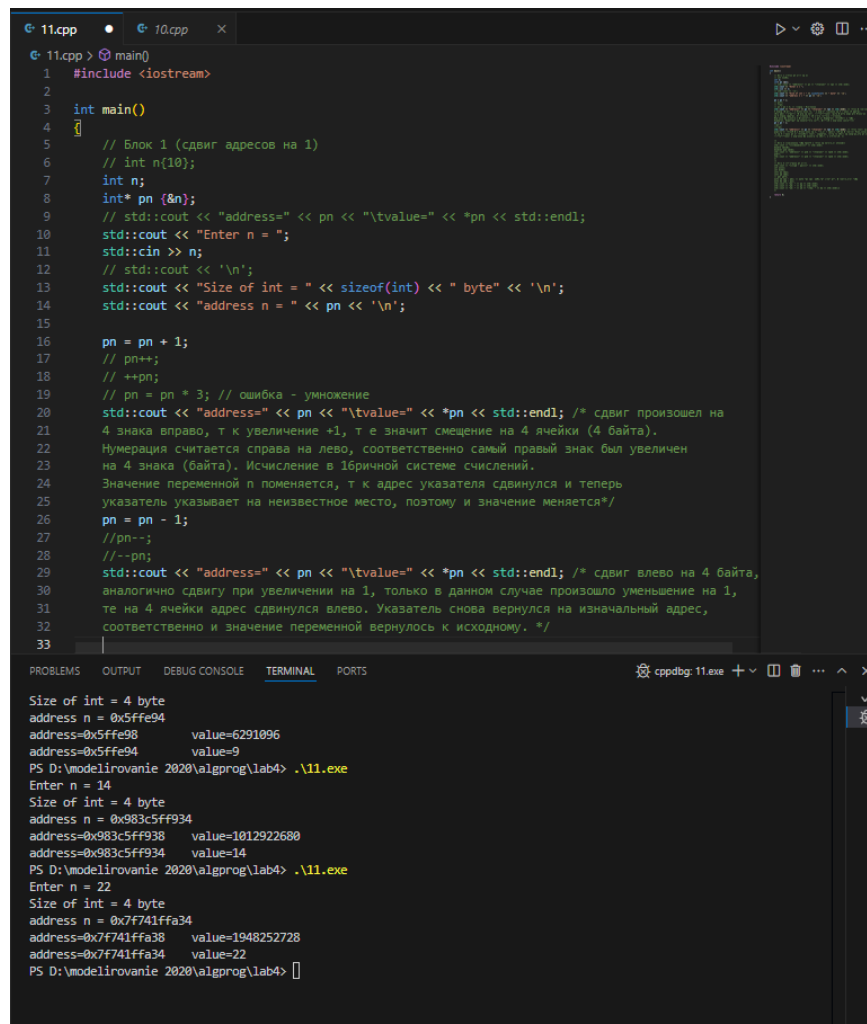
```
9.cpp > main()
1 #include <iostream>
2
3 int main()
4 {
5     // переменные разных типов
6     int a(3);
7     int b(11);
8     char c {'B'};
9     std::cout << c << '\n';
10
11     // формируем показатели для переменных
12     int* pa(&a);
13     int* pb(&b);
14     char* pc(&c); // указатель на символ
15
16     // комментарий 1 - std::cout << "pc=" << pc << std::endl; // ошибка
17
18     // комментарий 2
19     // формируем показатели int или void, чтобы получить адрес для char
20     int* pd {(int*)pc}; // указатель на int, чтобы вывести адрес
21     void* pv {(void*)pc}; // указатель на void, чтобы вывести адрес
22     std::cout << "pv=" << pv << std::endl;
23     std::cout << "pd=" << pd << std::endl;
24
25     // комментарий 3
26     // проверка возможности получения адреса при типах int
27     std::cout << "pa=" << pa << std::endl;
28     std::cout << "pb=" << pb << std::endl;
29     pb=pa;
30     std::cout << "pb (2)=" << pb << std::endl;
31     //pc=pb; // ошибка
32     //std::cout << "pc=" << pc << std::endl; // ошибка
33
34     return 0;
35 }
```

PS D:\modelirovanie 2020\algprog\lab4> & "c:\Users\Екатерина\.vscode\extensions\ms-vscode.cpptools-1.25.3-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe" "--stdin=Microsoft-MIEngine-In-4\ibals.rwl" "--stdout=Microsoft-MIEngine-Out-13\ldyn.ef0" "--stderr=Microsoft-MIEngine-Error-ykwtetp.s2p" "--pid=Microsoft-MIEngine-Pid-mdtFurd.orn" "--dbgExe=D:\modelirovanie 2020\algprog\mingw\ucrt64\bin\gdb.exe" "--interpreter=mi"
B
pv=0x5ffe6f
pd=0x5ffe6f
pa=0x5ffe74
pb=0x5ffe70
pb (2)=0x5ffe74
PS D:\modelirovanie 2020\algprog\lab4>

Рисунок 6. Код и результаты компиляции программы с согласованием типов данных для указателей.

Результаты упражнения 2.8. Указатели и арифметические операции.

На рисунке 7 программа с использованием итерации с указателем. В данной программе была определена переменная **n** и указатель на адрес данной переменной **pn**. Далее была произведена итерация, были протестированы варианты с префиксным и постфиксным инкрементом, а также декрементом. На скриншоте вариант итерации в формате **pn = pn + 1** и **pn = pn - 1**. Так же была опробована возможность умножения, но это невозможно, так как сама по себе операция сложения указателей это физически сдвиг стека на то или иное количество ячеек (байтов), в зависимости от архитектуры компьютера и типа данных переменных и указателей, например размер количества занимаемых ячеек указателем рассчитывается исходя из разрядности операционной системы (32-х или 64-х), если 32-х, тогда указатель занимает 4 байта, если 64-х, тогда 8 байтов. В данном случае, чтобы заведомо рассчитать сдвиг, для начала на консоль был выведен размер, который занимают переменные с типом `int` (4 байта). Далее уже были произведены арифметические операции и вывод адреса и значения переменной на консоль. В первом случае при увеличении указателя (адреса) на 1, сдвиг произошел на 4 знака вправо, так как увеличение +1, т е значит смещение на 4 ячейки (4 байта). Нумерация считается справа на лево, соответственно самый правый знак был увеличен на 4 знака (байта). Исчисление в 16ричной системе счислений. Значение переменной **n** поменяется, так как адрес указателя сдвинулся и теперь указатель указывает на неизвестное место, поэтому и значение меняется. В случае с уменьшением адреса переменной через указатель на 1, так как операция происходила после операции увеличения произошло следующее: сдвиг влево на 4 байта, аналогично сдвигу при увеличении на 1, только в данном случае произошло уменьшение на 1, те на 4 ячейки адрес сдвинулся влево. Указатель снова вернулся на изначальный адрес, соответственно и значение переменной вернулось к исходному.

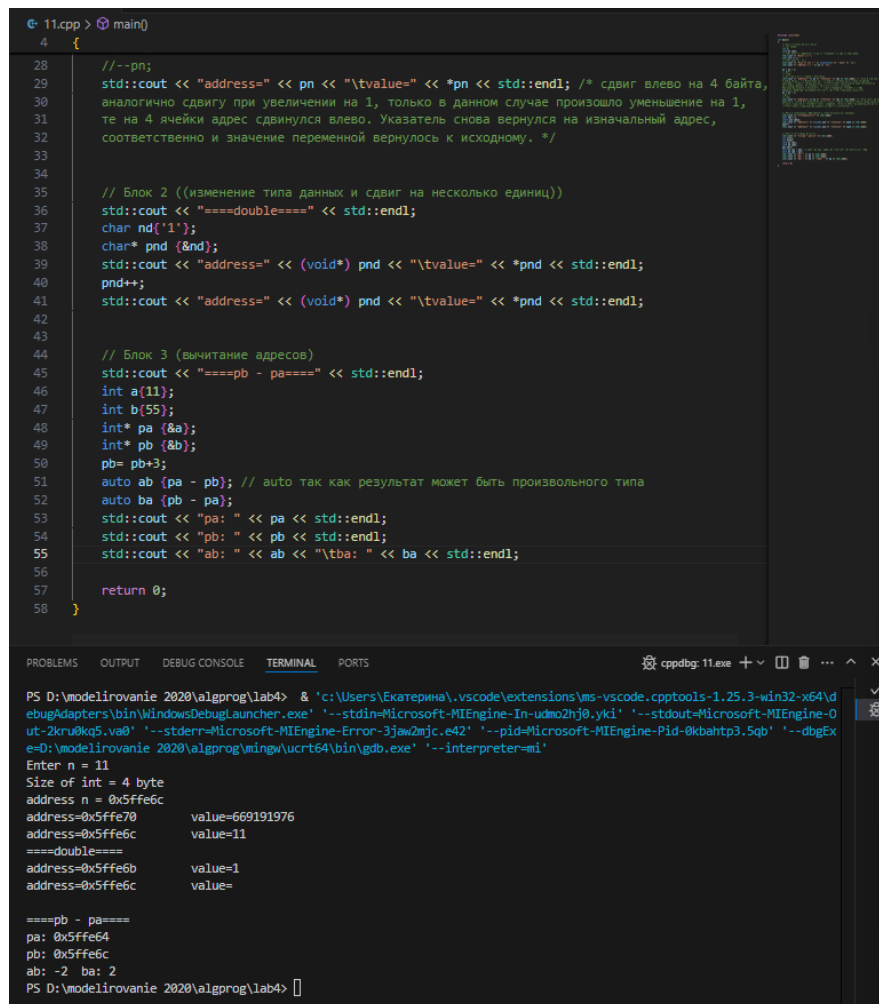


```
1  #include <iostream>
2
3  int main()
4  {
5      // Блок 1 (сдвиг адресов на 1)
6      int n(10);
7      int n;
8      int* pn {&n};
9      // std::cout << "address=" << pn << "\tvalue=" << *pn << std::endl;
10     std::cout << "Enter n = ";
11     std::cin >> n;
12     // std::cout << '\n';
13     std::cout << "Size of int = " << sizeof(int) << " byte" << '\n';
14     std::cout << "address n = " << pn << '\n';
15
16     pn = pn + 1;
17     // pn++;
18     // ++pn;
19     // pn = pn * 3; // ошибка - умножение
20     std::cout << "address=" << pn << "\tvalue=" << *pn << std::endl; /* сдвиг произошел на
21     4 знака вправо, т.к. увеличение +1, т.е. значит смещение на 4 ячейки (4 байта).
22     Нумерация считается справа на лево, соответственно самый правый знак был увеличен
23     на 4 знака (байта). Исчисление в 16ричной системе счисления.
24     Значение переменной n поменяется, т.к. адрес указателя сдвинулся и теперь
25     указатель указывает на неизвестное место, поэтому и значение меняется*/
26     pn = pn - 1;
27     //pn--;
28     //--pn;
29     std::cout << "address=" << pn << "\tvalue=" << *pn << std::endl; /* сдвиг влево на 4 байта,
30     аналогично сдвигу при увеличении на 1, только в данном случае произошло уменьшение на 1,
31     т.е. на 4 ячейки адрес сдвинулся влево. Указатель снова вернулся на изначальный адрес,
32     соответственно и значение переменной вернулось к исходному. */
33 }
```

Size of int = 4 byte
address n = 0x5ffe94
address=0x5ffe98 value=10
address=0x5ffe94 value=9
PS D:\modellirovanie 2020\algprog\lab4> .\11.exe
Enter n = 14
Size of int = 4 byte
address n = 0x983c5ff934
address=0x983c5ff938 value=1012922680
address=0x983c5ff934 value=14
PS D:\modellirovanie 2020\algprog\lab4> .\11.exe
Enter n = 22
Size of int = 4 byte
address n = 0x7f741ffa34
address=0x7f741ffa38 value=1948252728
address=0x7f741ffa34 value=22
PS D:\modellirovanie 2020\algprog\lab4>

Рисунок 7. Код и результаты компиляции программы с итерацией указателя.

На рисунке 8 программа с вычитанием адресов через указатели. В данном случае были введены 2 переменные **a** и **b**, а также указатели, указывающие на адреса данных переменных соответственно **pa** и **pb**. Далее была произведена итерация с увеличением адреса на 3 (т. е. на 12 байт/ смещение в памяти на 12 ячеек вправо). После чего была произведена операция вычитания адресов через указатели $ab \{pa - pb\}$; $ba \{pb - pa\}$. В данном случае вычитаются адреса одной переменной из другой, а на консоль выводится разница, чтобы ее вычислить, нужно получившийся результат поделить на 4 (адреса вычитаются с конца справа на лево, у всех переменных тип `int`, именно поэтому и производится в конце деление на 4, т.к. 1 переменная этого типа занимает 4 байта). Важно отметить, что операция сложения невозможна, так как сумма адресов будет очень большая, и не уместится в память.



```
28 //--pn;
29 std::cout << "address=" << pn << "\tvalue=" << *pn << std::endl; /* сдвиг влево на 4 байта,
30 аналогично сдвигу при увеличении на 1, только в данном случае произошло уменьшение на 1,
31 те на 4 ячейки адрес сдвинулся влево. Указатель снова вернулся на изначальный адрес,
32 соответственно и значение переменной вернулось к исходному. */
33
34
35 // Блок 2 ((изменение типа данных и сдвиг на несколько единиц))
36 std::cout << "===double===" << std::endl;
37 char nd{'1'};
38 char* pnd (&nd);
39 std::cout << "address=" << (void*) pnd << "\tvalue=" << *pnd << std::endl;
40 pnd++;
41 std::cout << "address=" << (void*) pnd << "\tvalue=" << *pnd << std::endl;
42
43
44 // Блок 3 (вычитание адресов)
45 std::cout << "===pb - pa====" << std::endl;
46 int a{11};
47 int b{55};
48 int* pa (&a);
49 int* pb (&b);
50 pb= pb+3;
51 auto ab {pa - pb}; // auto так как результат может быть произвольного типа
52 auto ba {pb - pa};
53 std::cout << "pa: " << pa << std::endl;
54 std::cout << "pb: " << pb << std::endl;
55 std::cout << "ab: " << ab << "\tba: " << ba << std::endl;
56
57 return 0;
58 }
```

```
PS D:\modelirovanie 2020\algprog\lab4> & 'c:\Users\Екатерина\.vscode\extensions\ms-vscode.cpptools-1.25.3-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-udmo2hj0.yki' '--stdout=Microsoft-MIEngine-0
ut-2kru0kq5.va0' '--stderr=Microsoft-MIEngine-Error-3jaw2mjc.e42' '--pid=Microsoft-MIEngine-Pid-0kbahtp3.5qb' '--dbgEx
e=0:\modelirovanie 2020\algprog\mingw\ucrt64\bin\gdb.exe' '--interpreter=mi'
Enter n = 11
Size of int = 4 byte
address n = 0x5ffe6c
address=0x5ffe70 value=669191976
address=0x5ffe6c value=11
===double===
address=0x5ffe6b value=1
address=0x5ffe6c value=
===pb - pa====
pa: 0x5ffe64
pb: 0x5ffe6c
ab: -2 ba: 2
PS D:\modelirovanie 2020\algprog\lab4>
```

Рисунок 8. Код и результаты компиляции программы с вычитанием адресов через указатели.

Результаты упражнения 2.9. Константы и указатели.

На рисунке 9 программа с использованием константного указателя и указателя на константу. В данной программе приведен пример разных возможных вариантов выводов константного указателя, указателя на константу и переменной, в зависимости от заданного типа данных. В случае с указателем на константу, всегда можно поменять адрес и значение переменной, независимо от того, что переменная или указатель принимают значение константы, но указатель не может изменяться (т.е. если есть переменная `a{20}` и указатель на константу `const int *pa{&a}`, то если мы присвоим `*pa = 40`, то нам программа выдаст ошибку). И наоборот в случае с константным указателем, может меняться все, кроме адреса.

```

1 // Блок 1. перем - var, указат - const =>
2 // изменить значение через указатель нельзя *pa =NO, но можно изменить адрес.
3 std::cout <<"===== var - const =====" << std::endl; // блок 1
4
5 int a {20};
6 const int *pa (&a);
7 const int b {45};
8
9 std::cout <<"a=" << a << "\tb=" << b << std::endl; // блок 1
10 std::cout <<"value *pa=" << *pa << std::endl; // value *pa=20
11 a = 30;
12 std::cout <<"value *pa=" << *pa << std::endl; // value *pa=30
13
14 // *pa = 40; // ошибка изменения значения
15 a = 20;
16 pa = &b; // указатель указывает на константу b
17 std::cout <<"*pa = " << *pa << std::endl; // *pa = 45
18 std::cout <<"a = " << a << std::endl; // a = 20 - переменная a не изменяется
19
20 // Блок 2. перем - const, указат - const =>
21 // изменить значение через указатель нельзя *pa =NO, но можно изменить адрес.
22 std::cout <<"===== const - const int =====" << std::endl; // блок 2
23
24 const int d {10};
25 const int *pd (&d); // указатель указывает на константу d
26 // *pd = 40; // ошибка изменения значения
27 pd = &b; // указатель указывает на константу b
28 std::cout <<"*pd = " << *pd << std::endl; // *pd = 45
29 std::cout <<"d = " << d << std::endl; // d = 10 - константа a не изменяется
30
31 // Блок 3. Константный указатель
32 // Блок 3.1 перем -int, указат - int const =>
33 // можно изменить значение через указатель *pf, нельзя изменить адрес pf =NO.
34 std::cout <<"===== const - (const-const) =====" << std::endl; // блок 3
35
36 // Константный указатель на переменную int f
37 int f {21};
38 int* const pf (&f);
39 std::cout << "value pf=" << *pf << std::endl; // value = 21
40 *pf = 25; // меняю значение по ссылке
41 std::cout << "value pf=" << *pf << std::endl; // value = 25
42 int f2 {32};
43 // pf = &b; // ошибка - недопустимая операция с адресом
44
45 // Константный указатель на константу const
46 // Блок 3.2 перем -const int, указат - int const =>
47 // нельзя изменить значение через указатель *pg =NO., нельзя изменить адрес pg =NO.
48 int g {31};
49 const int *const pg (&g);
50 std::cout << "value pg=" << *pg << "\tvalue g=" << g << std::endl;
51 // *pg = 41; // - ошибка - недопустимая операция со значением
52 int g2 {44};
53 // pg = &g2; // ошибка - недопустимая операция с адресом
54
55 return 0;

```

Рисунок 9. Код программы с указателем на константу и константным указателем.

```

1 #include <iostream>
2
3 int main()
4 {
5     // Блок 1. перем - var, указат - const =>
6     // изменить значение через указатель нельзя *pa =NO, но можно изменить адрес.
7     std::cout <<"===== var - const =====" << std::endl; // блок 1
8
9     int a {20};
10    const int *pa (&a);
11    const int b {45};
12
13    std::cout <<"a=" << a << "\tb=" << b << std::endl; // блок 1
14    std::cout <<"value *pa=" << *pa << std::endl; // value *pa=20
15    a = 30;
16    std::cout <<"value *pa=" << *pa << std::endl; // value *pa=30
17
18    // *pa = 40; // ошибка изменения значения
19    a = 20;
20    pa = &b; // указатель указывает на константу b
21    std::cout <<"*pa = " << *pa << std::endl; // *pa = 45
22    std::cout <<"a = " << a << std::endl; // a = 20 - переменная a не изменяется
23
24    // Блок 2. перем - const, указат - const =>

```

```

===== var - const =====
a=20    b=45
value *pa=20
value *pa=30
*pa = 45
a = 20
===== const - const int =====
*pd = 45
d = 10
===== const - (const-const) =====
value pf=21
value pf=25
value pg=31    value g=31
PS D:\modelirovanie 2020\algprog\lab4>

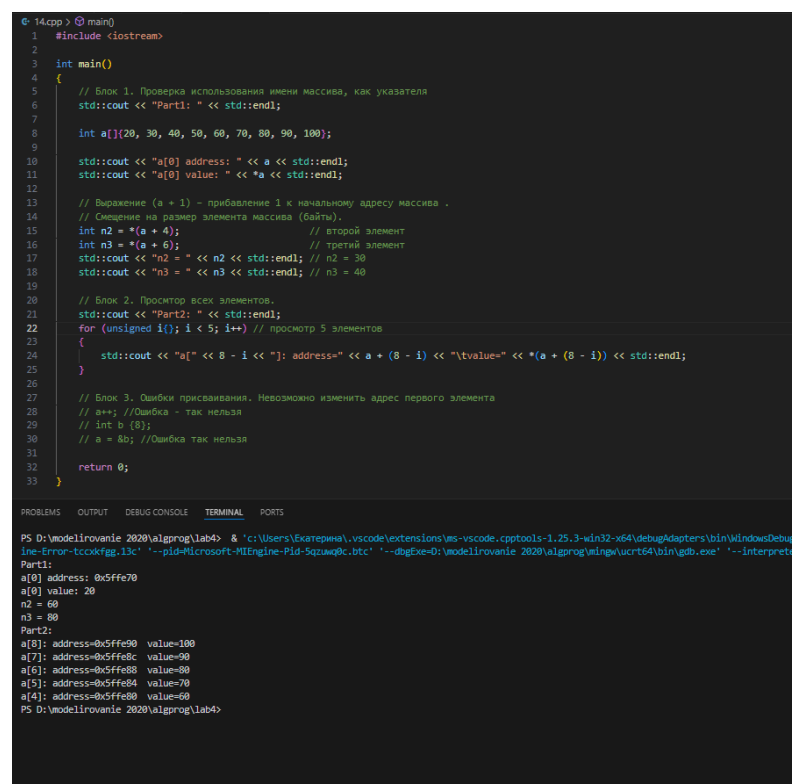
```

Рисунок 10. Результаты компиляции программы с указателем на константу и константным указателем.

Часть 3. Указатели и массивы.

Результаты упражнения 3.1. Проверка использования имени массива, как указателя.

На рисунке 11 программа с проверкой использования имени массива, как указателя. В данной программе можно увидеть, что имя массива по сути является адресом его первого элемента. А так же при форме записи (при заданном массиве `a[9]{{}}`) `int n2 = *(a+1)`, была введена переменная с указателем на 2 элемент массива, т. е. благодаря такой форме записи можно выводить отдельные элементы массива на консоль. Так же благодаря такому методу можно написать цикл для вывода всего массива. Для вывода последних 5 элементов массива в блоке 2 использовался «реверсивный» метод вывода (от обратного).



```
14cpp > main()
1 #include <iostream>
2
3 int main()
4 {
5     // Блок 1. Проверка использования имени массива, как указателя
6     std::cout << "Part1: " << std::endl;
7
8     int a[] {20, 30, 40, 50, 60, 70, 80, 90, 100};
9
10    std::cout << "a[0] address: " << a << std::endl;
11    std::cout << "a[0] value: " << *a << std::endl;
12
13    // Выражение (a + 1) - прибавление 1 к начальному адресу массива .
14    // Смещение на размер элемента массива (байты).
15    int n2 = *(a + 4); // второй элемент
16    int n3 = *(a + 6); // третий элемент
17    std::cout << "n2 = " << n2 << std::endl; // n2 = 30
18    std::cout << "n3 = " << n3 << std::endl; // n3 = 40
19
20    // Блок 2. Просмотр всех элементов.
21    std::cout << "Part2: " << std::endl;
22    for (unsigned i{}; i < 5; i++) // просмотр 5 элементов
23    {
24        std::cout << "a[" << 8 - i << "]: address=" << a + (8 - i) << "\tvalue=" << *(a + (8 - i)) << std::endl;
25    }
26
27    // Блок 3. Ошибка присваивания. Невозможно изменить адрес первого элемента
28    // a++; //Ошибка - так нельзя
29    // int b {8};
30    // a = &b; //Ошибка так нельзя
31
32    return 0;
33 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

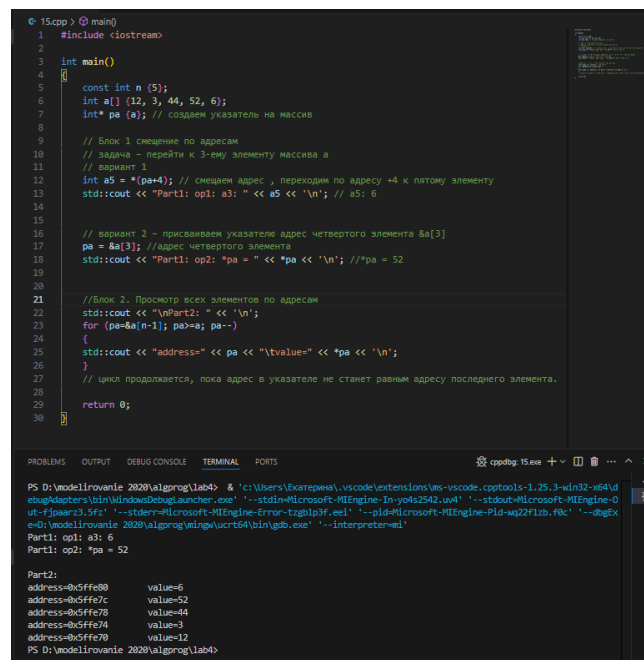
```
PS D:\modelirovanie 2020\alprog\lab4> & 'c:\Users\Екатерина\.vscode\extensions\ms-vscode.cpptools-1.25.3-win32-x64\debugAdapters\bin\WindowsDebug
Line-Error-tccxfgg.13c' '--pid=Microsoft-MiEngine-Pid-Squw0hc.btc' '--dbgExe=D:\modelirovanie 2020\alprog\wingw\ucr64\bin\gdb.exe' '--interpret
Part1:
a[0] address: 0x5ffe70
a[0] value: 20
n2 = 30
n3 = 40
Part2:
a[8]: address=0x5ffe90 value=100
a[7]: address=0x5ffe8c value=90
a[6]: address=0x5ffe88 value=80
a[5]: address=0x5ffe84 value=70
a[4]: address=0x5ffe80 value=60
PS D:\modelirovanie 2020\alprog\lab4>
```

Рисунок 11. Код и результаты компиляции программы с проверкой использования имени массива, как указателя.

Результаты упражнения 3.2. Указатели на массивы.

На рисунке 12 программа с примером использования указателя на массив. В данном случае был приведен пример варианта со смещением по адресам (блок 1, состоит из двух вариантов, где в первом варианте введенной переменной

присваивается значение через указатель на переход по адресу +2 к третьему элементу, а во втором варианте используется адрес, который указывает на третий элемент). А так же в данной программе в блоке 2 был произведен просмотр всех элементов по адресам, чтобы реализовать просмотр с конца 5 элементов, в инициализации цикла for указателю был присвоен адрес последнего элемента массива, а в условиях указатель должен принимать адрес больше или равный до адреса первого элемента массива (если нужно ограничение, например последние 3 элемента, тогда это реализуется через адрес с указанием на величину до 3 элемента с конца).



```
1 #include <iostream>
2
3 int main()
4 {
5     const int n (5);
6     int a[] (12, 3, 44, 52, 6);
7     int* pa (a); // создаем указатель на массив
8
9     // Блок 1. Смещение по адресам
10    // задача - перейти к 3-ему элементу массива a
11    // вариант 1
12    int a5 = *(pa+4); // смещаем адрес , переходим по адресу +4 к пятому элементу
13    std::cout << "Part1: op1: a3: " << a5 << '\n'; // a5: 6
14
15    // вариант 2 - присваиваем указателю адрес четвертого элемента &a[3]
16    pa = &a[3]; //адрес четвертого элемента
17    std::cout << "Part1: op2: *pa = " << *pa << '\n'; // *pa = 52
18
19    //Блок 2. Просмотр всех элементов по адресам
20    std::cout << "\nPart2: " << '\n';
21    for (pa=&a[n-1]; pa>=a; pa--)
22    {
23        std::cout << "address=" << pa << "\tvalue=" << *pa << '\n';
24    }
25    // цикл продолжается, пока адрес в указателе не станет равным адресу последнего элемента.
26
27    return 0;
28 }
29
30
```

PS D:\modelirovanie 2020\algprog\lab4> g++ "c:\Users\katerina\.vscode\extensions\ms-vscode.cpptools-1.25.3-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe" --stdinMicrosoft-MIEngine-In-y04s2542.uv" --stdoutMicrosoft-MIEngine-Out-y04s2542.uv" --diagExe=D:\modelirovanie 2020\algprog\wingw\ucrt64\bin\gdb.exe' '-interpreter=mi'

Part1: op1: a3: 6
Part1: op2: *pa = 52

Part2:
address=0x5ffe80 value=6
address=0x5ffe7c value=52
address=0x5ffe78 value=44
address=0x5ffe74 value=3
address=0x5ffe70 value=12

Рисунок 12. Код и результаты компиляции программы с указателями на массивы.

Результаты упражнения 3.4. Перебор элементов многомерного массива с использованием указателей.

На рисунке 13 программа с примером перебора элементов многомерного массива с использованием указателей. В данной программе был приведен пример вывода (прочтения) двумерного массива через указатели двумя способами. В первом случае через контроль значения адреса последнего элемента, во втором через расчет на основе контроля общего числа элементов. И в том и в другом случае есть ряд особенностей в сравнении с работой только с выводом массива: вложенный цикл не используется, так как в этом нет необходимости, перебор и вывод элементов на консоль выходит построчно за счет использования указателя на массив, но при этом есть зависимость в выводе, изначально необходимо ввести переменные с

количеством строк и столбцов (до начала цикла), после чего внутри цикла, необходимо сделать условие после вывода элементов через указатель, с проверкой на остаток от деления инициализированной переменной в цикле for от количества столбцов, при нуле выполняется условия перехода на новую строку. Это работает таким образом, что если в массиве, например, 4 столбца, то получается каждый 4 элемент будет делиться на 4 без остатка.

```

1 #include <iostream>
2
3 int main()
4 {
5     // можно ввести из консоли nr (число строк), nc (число столбцов)
6     int ab[3][4] = { {1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12} };
7     unsigned int nr = sizeof(ab)/sizeof(ab[0]); // число строк
8     unsigned int nc = sizeof(ab[0])/sizeof(ab[0][0]); // число столбцов
9
10    // Блок 1. Расчет на основе контроля значения адреса последнего элемента end
11    int* end(ab[0] + nr * nc - 1); // указатель на самый последний элемент 0 + 3 * 4 - 1 = 11
12    int* pa(ab[0]); // указатель на первый элемент
13    for( unsigned i(1); pa <= end; pa++, i++)
14    {
15        std::cout << *pa << "\t";
16        // если остаток от целочисленного деления равен 0,
17        // переходим на новую строку
18        if(i%nc == 0)
19        {
20            std::cout << std::endl;
21        }
22    }
23
24    // Блок 2. Расчет на основе контроля общего числа элементов
25    std::cout << "----- Part 2-----" << std::endl;
26
27    const unsigned cnt(nc * nr); // общее количество элементов, умножение
28    pa = ab[0]; // указатель на первый элемент первого массива
29    for(unsigned i(1); i <= cnt; pa++, i++)
30    {
31        std::cout << *pa << "\t";
32        // если остаток от целочисленного деления равен 0,
33        // переходим на новую строку
34        if(i%nc == 0)
35        {
36            std::cout << std::endl;
37        }
38    }
39
40    return 0;
41 }
42
43

```

Terminal Output:

```

PS D:\modelirovanie 2020\alprog\lab4> g++ 17.exe & 'c:\Users\Exaterepna\vscode\extensions\ms-vscode.cpptools-1.25.3-win32-x64\debugadapters\bin\WindowsDebugLauncher.exe' --stdin=Microsoft-MIEngine-In-0a080kz.zdi' --stdout=Microsoft-MIEngine-0
ut-ve5tgsso.pto' --stderr=Microsoft-MIEngine-Error-sei4wkw.2sh' --pid=Microsoft-MIEngine-Pid-231zovlw.mrh' --dbgEx
e=D:\modelirovanie 2020\alprog\mingw\vcrt64\bin\gdb.exe' --interpreter=mi1
1 2 3 4
5 6 7 8
9 10 11 12
----- Part 2-----
1 2 3 4
5 6 7 8
9 10 11 12
PS D:\modelirovanie 2020\alprog\lab4>

```

Рисунок 13. Код и результаты компиляции программы с перебором элементов многомерного массива с использованием указателей.

Контрольная задача.

Цель

Цель – наработать навык повторения материала по использованию массивов, использованию циклов и операторов условий, а также приобретение новых навыков использования указателей и определение длительности выполнения программы.

Выбор варианта:

N-номер в журнале - 8

Op(S) (номер оценочного выражения $[1,5]$) = $N \bmod 5 + 1 = 4$ (см рис 14)

$$4) S = \left(\frac{\sqrt[3]{(y+z^x)} (\sin x + \cos z)^2}{y(\sin(zx))^2 \ln(e^x + e^z)} + \frac{\sqrt[2]{(y+x^z)} (\sin x)^2}{\lg(y+x^z+z^x)} \right)$$

Рисунок 14. Оценочное выражение для расчета оценочного массива

Задание:

Написать 2 программы. Одна программа использует только массивы, вторая использует указатели при работе с элементами:

1) Задаем номер теста **t (ввод из консоли)**. Создаем двумерный массив размерностью **[n] [m]**. Значения **n, m** вводим либо вручную, либо **из консоли**. (Учесть, что всего может быть более 1000000 элементов, например, [2000][3000] и более). Попробуйте определить предел (в этом случае run-time выдаст ошибку по нехватке памяти или ошибки по данным). При вводе значений **n, m** вручную необходимо использовать динамические массивы.

2) Каждому элементу массива присваиваем значение функции (используем функцию из **Задания лабораторной работы 1 (вариант из работы 1)**). Значения (x, y, z) задаем псевдослучайным образом для каждого значения элемента, например, на основе индекса цикла.

3) Вычисляем максимум, минимум и среднее значение всех элементов для новых значений. Это просто нагрузка на процессор.

4) Используем код шаблона для определения времени выполнения программы.

5) Формируем итоговую таблицу (можно вручную, не автоматически)

6) Выводы о времени выполнения для программ (разное, или не изменилось).

Листинг программ:

1. Листинг программы с использованием только массивов (в программе есть комментарии, а также закомментированные строки – они были добавлены, чтобы осуществлять проверку правильной работы команд и операций через вывод результатов на консоль)

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <cmath>
```

```
#include <limits>
```

```
int main()
```

```
{
```

```
    std::cout << "Group, Name: 2346, E. Isaeva, N=8" << '\n'; // вводная часть, которая выводится на
```

консоль

```
    std::cout << "Task - Array and Pointers" << '\n';
```

```
    std::cout << "Type: <1) Array, 2) Array and pointers> This is Program 1) Array" << '\n';
```

```
    std::cout << "Options of equation: Op(S)=4 (Op(S) = Nmod5+1)" << '\n';
```

```
    int i = 1;
```

```
    while (i < 5) // цикл для повторения тестов, итерация i++
```

```
    {
```

```
        unsigned n, m, total;
```

```
        std::cout << "\nTest t = " << i << "." << ' ';
```

```
        std::cout << "Input: n (rows) = ";
```

```
        std::cin >> n;
```

```
        std::cout << "Input: n (rows) = " << n << ", m(columns) = ";
```

```
        std::cin >> m;
```

```
        total = m * n;
```

```
        std::cout << "Input: n (rows) = " << n << ", m(columns) = " << m << ", total = " << total << " (elements)"
```

```
<< '\n';
```

```
        unsigned int start_time = clock(); // проверка времени выполнения одного цикла
```

```
        unsigned **nm{new unsigned *[n] {}}; // определение динамического двумерного массива
```

```
        for (unsigned q{}; q < n; q++)
```

```
        {
```

```
            nm[q] = new unsigned[m]{};
```

```
        }
```

```
        unsigned sum = 0; // определение переменной sum для вычисления суммы всех элементов
```

массива

```
        unsigned minVal = nm[0][0]; // определение переменной minVal для вычисления минимального
```

элемента массива

```
        unsigned maxVal = nm[0][0]; // определение переменной maxVal для вычисления максимального
```

элемента массива

```

for (unsigned i = 0; i < n; i++)
{
    for (unsigned j = 0; j < m; j++)
    {
        nm[i][j] = rand(); // рандомное вычисление значений элементов массива
        // std::cout << nm[i][j] << '\t'; проверка правильного вывода массива и расчет переменных
        sum += nm[i][j]; // определение суммы, максимального и минимального значения элементов
массива

        if (nm[i][j] < minVal)
            minVal = nm[i][j];
        if (nm[i][j] > maxVal)
            maxVal = nm[i][j];
    }
    // std::cout << '\n';
}

```

```

std::cout << "\n1. Random array: max= " << maxVal << '\t'; // вывод максимального элемента массива
std::cout << "min= " << minVal << '\t'; // вывод минимального элемента массива
std::cout << "average= " << sum; // вывод суммы всех элементов */

```

```

sum = 0; // определение переменной sum для вычисления суммы всех элементов массива
minVal = nm[0][0]; // определение переменной minVal для вычисления минимального элемента
массива

```

```

maxVal = nm[0][0]; // определение переменной maxVal для вычисления максимального элемента
массива

```

// цикл для формирования массива по значениям введенных с консоли столбцов и строк

```

for (unsigned q = 0; q < n; q++)
{
    for (unsigned k = 0; k < m; k++)
    {
        // рандомное определение значения переменной x,y,z (20% от диапазона положительных
чисел включая 0)

        float x = rand() % 10;
        float y = rand() % 10;
        float z = rand() % 10;
        float S1, S2, numer1, numer2, denom1, denom2; // определение переменных суммы первого
слагаемого

```

```
// и второго слагаемого, а так же числителей и знаменателей соответственно
```

```
// расчет ранее обозначенных переменных
```

```
numer1 = (cbrt(y + (pow(z, x)))) * (pow((sin(x) + cos(z)), 2));
```

```
denom1 = y * (pow((sin(z * x)), 2)) * (log(((exp(x)) + (exp(z)))));
```

```
numer2 = (sqrt(y + (pow(x, z)))) * (pow((sin(x)), 2));
```

```
denom2 = log(y + (pow(x, z) + (pow(z, x))));
```

```
S1 = numer1 / denom1;
```

```
S2 = numer2 / denom2;
```

```
float S = S1 + S2; // расчет общей суммы
```

```
unsigned value = static_cast<unsigned>(S); // присвоение переменной S тип данных unsigned
```

```
int
```

```
nm[q][k] = value; // присвоение элементу массива значение суммы арифм операции
```

```
// std::cout << nm[q][k] << '\t'; проверка правильного вывода массива и расчет переменных
```

```
sum += nm[q][k];
```

```
if (nm[q][k] < minVal)
```

```
    minVal = nm[q][k];
```

```
if (nm[q][k] > maxVal)
```

```
    maxVal = nm[q][k];
```

```
}
```

```
// std::cout << '\n';
```

```
}
```

```
// Вывод суммы, максимального значения и минимального
```

```
std::cout << "\n2. Estimated array: max= " << maxVal << ' '; // вывод максимального элемента
```

```
массива
```

```
std::cout << "min= " << minVal << ' ';
```

```
// вывод минимального элемента массива
```

```
std::cout << "average= " << sum;
```

```
// вывод суммы всех элементов
```

```
// очистка памяти
```

```
for (unsigned q = 0; q < n; q++)
```

```
{
```

```
    delete[] nm[q];
```

```
}
```

```
delete[] nm;
```

```

    unsigned int end_time = clock();
    unsigned int search_time = end_time - start_time;
    std::cout << "\n3. Elapsed time: time = " << search_time << '\n';

    i++; // итерация
}

/* unsigned int end_time = clock();
unsigned int search_time = end_time - start_time;
std::cout << " 3. Elapsed time: time = " << search_time; */
return 0;
}

```

2. Листинг программы с использованием указателей для массива (в программе есть комментарии, а также закомментированные строки – они были добавлены, чтобы осуществлять проверку правильной работы команд и операций через вывод результатов на консоль)

```

#include <iostream>
#include <cstdlib>
#include <ctime>
#include <cmath>
#include <limits>

int main()
{
    std::cout << "Group, Name: 2346, E. Isaeva, N=8" << '\n'; // вводная часть, которая выводится на
    консоль

    std::cout << "Task - Array and Pointers" << '\n';
    std::cout << "Type: <1) Array, 2) Array and pointers> This is Program 2) Array and pointers" << '\n';
    std::cout << "Options of equation: Op(S)=4 (Op(S) = Nmod5+1)" << '\n';

    int i = 1;
    while (i < 5) // цикл для повторения тестов, итерация i++
    {
        unsigned n, m, total;

        std::cout << "\nTest t = " << i << "." << ' ';
        std::cout << "Input: n (rows) = ";
    }
}

```

```

std::cin >> n;
std::cout << "Input: n (rows) = " << n << ", m(columns) = ";
std::cin >> m;
total = m * n;
std::cout << "Input: n (rows) = " << n << ", m(columns) = " << m << ", total = " << total << "
(elements)" << "\n";

unsigned int start_time = clock();    // проверка времени выполнения одного цикла
unsigned *nm_linear{new unsigned[n * m]{}; // определение динамического двумерного
массива
unsigned **nm{new unsigned *[n] {}};    // определение динамического двумерного массива
for (unsigned q{}; q < n; q++)
{
    nm[q] = &nm_linear[m * q];
}

// Random
unsigned int *end{nm[0] + n * m - 1}; // указатель на самый последний элемент 0 + 3 * 4 - 1 =
11
unsigned int *pa{nm[0]};    // указатель на первый элемент
*pa = rand() % 10;
unsigned minVal = *pa; // определение переменной minVal для вычисления минимального
элемента массива
unsigned maxVal = *pa; // определение переменной maxVal для вычисления максимального
элемента массива
unsigned sum = *pa; // определение переменной sum для вычисления суммы всех элементов
массива
unsigned av = *pa;    // определение переменной av для вычисления среднего значения
элементов массива

// std::cout << *pa << "\t";
pa++;
for (unsigned i{2}; pa <= end; pa++, i++)
{
    *pa = rand();
    sum += *pa; // определение суммы, максимального и минимального значения элементов
массива
    if (*pa < minVal)

```

```

        minVal = *pa;
    if (*pa > maxVal)
        maxVal = *pa;
    // std::cout << *pa << "\t";
    // если остаток от целочисленного деления равен 0,
    /* переходим на новую строку
    if (i % m == 0)
    {
        std::cout << std::endl;
    }*/
}

av = sum / total;

std::cout << "\n1. Random array: max= " << maxVal << ' '; // вывод максимального элемента
массива

std::cout << "min= " << minVal << ' '; // вывод минимального элемента массива
std::cout << "average= " << av; // вывод суммы среднего значения элементов
массива

// Estimated OS(S)
unsigned int *end2{nm[0] + n * m - 1}; // указатель на самый последний элемент 0 + 3 * 4 - 1 =
11
unsigned int *pa2{nm[0]}; // указатель на первый элемент
// рандомное определение значения переменной x,y,z (20% от диапазона положительных
чисел включая 0)
float x = rand() % 10;
float y = rand() % 10;
float z = rand() % 10;
float S1, S2, numer1, numer2, denom1, denom2; // определение переменных суммы первого
слагаемого
// и второго слагаемого, а так же числителей и знаменателей соответственно
// std::cout << x << y << z << '\n';

// расчет ранее обозначенных переменных
numer1 = (cbrt(y + (pow(z, x)))) * (pow((sin(x) + cos(z)), 2));
denom1 = y * (pow((sin(z * x)), 2)) * (log(((exp(x)) + (exp(z)))));
numer2 = (sqrt(y + (pow(x, z)))) * (pow((sin(x)), 2));
denom2 = log(y + (pow(x, z) + (pow(z, x))));

```

```

S1 = numer1 / denom1;
S2 = numer2 / denom2;

float S = S1 + S2; // расчет общей суммы
// std::cout << S << '\n';

unsigned value = static_cast<unsigned>(S); // присвоение переменной S тип данных unsigned
int
// std::cout << value << '\n';
*pa2 = value; // присвоение элементу массива значение суммы арифм операции
minVal = *pa2; // определение переменной minVal для вычисления минимального элемента
массива
maxVal = *pa2; // определение переменной maxVal для вычисления максимального элемента
массива
sum = *pa2; // определение переменной sum для вычисления суммы всех элементов массива
av = *pa2;

// std::cout << *pa2 << "\t";
pa2++;
// цикл для формирования массива по значениям введенных с консоли столбцов и строк
for (unsigned i{2}; pa2 <= end; pa2++, i++)
{
    // случайное определение значения переменной x,y,z (20% от диапазона положительных
чисел включая 0)
    x = rand() % 10;
    y = rand() % 10;
    z = rand() % 10;

    // расчет ранее обозначенных переменных
    numer1 = (cbrt(y + (pow(z, x)))) * (pow((sin(x) + cos(z)), 2));
    denom1 = y * (pow((sin(z * x)), 2)) * (log(((exp(x)) + (exp(z)))));
    numer2 = (sqrt(y + (pow(x, z)))) * (pow((sin(x)), 2));
    denom2 = log(y + (pow(x, z) + (pow(z, x))));
    S1 = numer1 / denom1;
    S2 = numer2 / denom2;

    float S = S1 + S2; // расчет общей суммы
    unsigned value = static_cast<unsigned>(S); // присвоение переменной S тип данных unsigned
int

```

```

        *pa2 = value;
        sum += *pa2; // определение суммы, максимального и минимального значения элементов
массива
        if (*pa2 < minVal)
            minVal = *pa2;
        if (*pa2 > maxVal)
            maxVal = *pa2;
        /* std::cout << *pa2 << "\t";
        // если остаток от целочисленного деления равен 0,
        // переходим на новую строку
        if (i % m == 0)
        {
            std::cout << std::endl;
        } */
    }

    // Вывод суммы, максимального значения и минимального
    av = sum / total;
    std::cout << "\n2. Estimated array: max= " << maxVal << ' '; // вывод максимального элемента
массива
    std::cout << "min= " << minVal << ' '; // вывод минимального элемента массива
    std::cout << "average= " << av; // вывод среднего значения элементов массива

    // очистка памяти
    delete[] nm;
    delete[] nm_linear;

    unsigned int end_time = clock();
    unsigned int search_time = end_time - start_time;
    std::cout << "\n3. Elapsed time: time = " << search_time << '\n';

    i++; // итерация
}

/* unsigned int end_time = clock();
unsigned int search_time = end_time - start_time;
std::cout << " 3. Elapsed time: time = " << search_time; */

```

```
    return 0;
}
```

На рисунке 15 часть кода программы с использованием массивов.

На рисунке 16 результаты компиляции программы с использованием массивов.

На рисунке 17 часть кода программы с использованием указателей при работе с элементами.

На рисунке 18 результаты компиляции программы с использованием указателей при работе с элементами.

```
18.cpp x 18.cpp
1 #include <iostream>
2 #include <cstdlib>
3 #include <ctime>
4 #include <cmath>
5 #include <limits>
6
7 int main()
8 {
9     // unsigned int start_time = clock(); // проверка времени выполнения программы
10
11     std::cout << "Group, Name: 2346, E. Isaeva, №8" << "\n"; // заголовок, который выводится на консоль
12     std::cout << "Task - Array and Pointers" << "\n";
13     std::cout << "Type: (1) Array, (2) Array and pointers This is Program 1 Array" << "\n";
14     std::cout << "Options of equation: Op(5)=4 (Op(5) = Nmod5+1)" << "\n";
15
16     int i = 1;
17     while (i < 5) // цикл для повторения тестов, итерация i++
18     {
19         unsigned n, m, total;
20
21         std::cout << "nTest t = " << i << ", " << " ";
22         std::cout << "Input: n (rows) = ";
23         std::cin >> n;
24         std::cout << "Input: m (columns) = " << n << ", m(columns) = ";
25         std::cin >> m;
26         total = n * m;
27         std::cout << "Input: n (rows) = " << n << ", m(columns) = " << m << ", total = " << total << " (elements)" << "\n";
28
29         unsigned int start_time = clock(); // проверка времени выполнения одного цикла
30         unsigned **nm = new unsigned*[n](); // определение динамического двумерного массива
31         for (unsigned q = 0; q < n; q++)
32         {
33             nm[q] = new unsigned[m]();
34         }
35
36         unsigned sum = 0; // определение переменной sum для вычисления суммы всех элементов массива
37         unsigned minVal = nm[0][0]; // определение переменной minVal для вычисления минимального элемента массива
38         unsigned maxVal = nm[0][0]; // определение переменной maxVal для вычисления максимального элемента массива
39
40         for (unsigned i = 0; i < n; i++)
41         {
42             for (unsigned j = 0; j < m; j++)
43             {
44                 nm[i][j] = rand(); // случайное вычисление значений элементов массива
45                 // std::cout << nm[i][j] << "\t"; // проверка правильного вывода массива и расчет переменных
46                 sum += nm[i][j]; // определение суммы, максимального и минимального значения элементов массива
47                 if (nm[i][j] < minVal)
48                     minVal = nm[i][j];
49                 if (nm[i][j] > maxVal)
50                     maxVal = nm[i][j];
51             }
52             // std::cout << "\n";
53         }
54
55         std::cout << "n1. Random array: max = " << maxVal << " "; // вывод максимального элемента массива
56         std::cout << "min = " << minVal << " "; // вывод минимального элемента массива
57         std::cout << "average = " << sum; // вывод суммы всех элементов */
58
59         sum = 0; // определение переменной sum для вычисления суммы всех элементов массива
```

Рисунок 15. Результаты компиляции программы с использованием массивов.

Итоговая таблица

В таблице 1 приведена итоговая таблица, сформированная исходя из результатов работы двух программ с одной задачей, но разными методами решения.

Номер эксперимента (t)	Число элементов в массиве	Время выполнения программы, ms	
		Для массива без указателей	Для массива с указателями
1	1000 = [100][10]	1	1
2	10000 = [100][100]	7	7
3	1000000 = [1000] [1000]	585	535
4	10000000 = [10000] [1000]	5656	4750

Выводы по результатам контрольного задания:

В результате выполненного контрольного задания, где было проведено сравнение между двумя программами с одинаковой задачей, но разным вариантом решения (в первом варианте используя только массивы, во втором – используются указатели при работе с элементами), можно сказать, что при относительно небольших размерах массива (примерно до 1000 на 1000) и нагрузкой в виде расчета минимального, максимального и среднего значений элементов массива, разница во времени выполнения одного цикла между двумя вариантами решения задачи либо отсутствует, либо незначительна. Но начиная с работы с массивами с 1000 на 1000 (кол-ом эл-ов примерно от 1000000) значение по времени отличается, и чем больше размер массива, тем больше разница во времени между двумя методами. Быстрее работает цикл с использованием указателей.

Вывод

В результате выполнения лабораторной работы были изучены, проанализированы и выполнены упражнения и задания к ним из части 1 (Ссылки) 1.2 – 1.3; части 2 (Указатели) 2.1 – 2.10; части 3 (Указатели и ссылки) 3.1 – 3.6. Были освоены навыки использования констант и ссылок, рассмотрены особенности

использования констант, а также использования ссылок для оптимизации циклов. Были освоены навыки работы с указателями: верный формат того, как надо задавать указатели, то как необходимо присваивать адрес переменной указателю, а также были освоены навыки вывода адреса переменной через указатель, вывод адреса указателя и вывод значения переменной через указатель. В работе также рассматривались константы и указатели (константный указатель и указатель на константу, и варианты использования), а также были получены навыки использования указателей на одномерные и двумерные массивы. После выполненных упражнений, было выполнено задание для закрепления навыков работы с ссылками, указателями и массивами. В задании так же был освоен навык работы с двумерными динамическими массивами, как через указатели, так и напрямую без их использования.